CLASES DE LOGICA DE PROGRAMACION BASICA DESDE CERO

VECTORES Y MATRICES

Sebastian Guevara Sanchez

Santiago de Cali – Colombia

2025 Septiembre 10

Tabla de contenido

ARREGLOS o VECTORES 3

Tipos de arreglos 3

Ejemplo de arreglo 3

MATRICES 5

Tipo de matices 6

Matriz por forma: 6

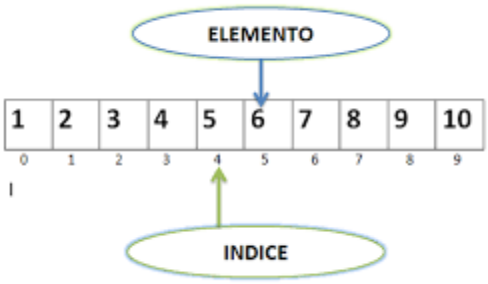
Matriz por valores: 7

Ejemplos de matrices 7

ARREGLOS o VECTORES

Un arreglo es un grupo de variables organizadas en un vector o una matriz, donde cada posición o también llamado variable tienen nombre e información interna.

En la siguiente imagen, se muestra un arreglo con posiciones (índice) de 0 al 9 y dentro de cada posición un valor (elemento) que en este caso van de 1 a 10.



Tipos de arreglos

Arreglos unidimensionales.

Arreglos multidimensionales.

Arreglo con múltiples subíndices.

Ejemplo de arreglo

En la siguiente imagen, podemos ver la creación de un arreglo en el lenguaje de programación PHP, este arreglo se llama $nombres y tiene dentro 3 nombres diferentes, y en las siguientes líneas se muestra una forma de imprimir cada dato dentro del arreglo con el nombre de la lista y su índice o posición.

```php
<?php
$nombres = ["Ana", "Bernardo", "Carmen"];

print "<p>$nombres[1]</p>\n";
print "<p>$nombres[0]</p>\n";
?>
```

En la siguiente imagen se ve la creación en PHP con las posiciones en texto y su información en números, luego con un print se muestra em pantalla la información de una posición en el arreglo.

```php
<?php
$edades = ["Andrés" => 20, "Bárbara" => 19, "Camilo"
=> 17];

print "<p>Bárbara tiene $edades[Bárbara] años</p>\n";
?>
```

Bárbara tiene 19 años

En la siguiente imagen, se muestra como se puede crear un arreglo, luego de importar la librería numpy y también podemos ver unos ejemplos de cómo se usa la función letters de numpy y con el print se muestra en pantalla.

matrices.py

```python
import numpy as np
letters = np.array([1, 3, 5, 7, 9, 7, 5])

# 3rd hasta el 5th elemento
print(letters[2:5])        # Salida: [5, 7, 9]

# 1st hasta el 4th elemento
print(letters[:-5])        # Salida: [1, 3]

# 6th al último elemento
print(letters[5:])         # Salida:[7, 5]

# 1st al último elemento
print(letters[:])          # Salida:[1, 3, 5, 7, 9, 7, 5]

# reverso de la lista
print(letters[::-1])          # Salida:[5, 7, 9, 7, 5, 3, 1]
```

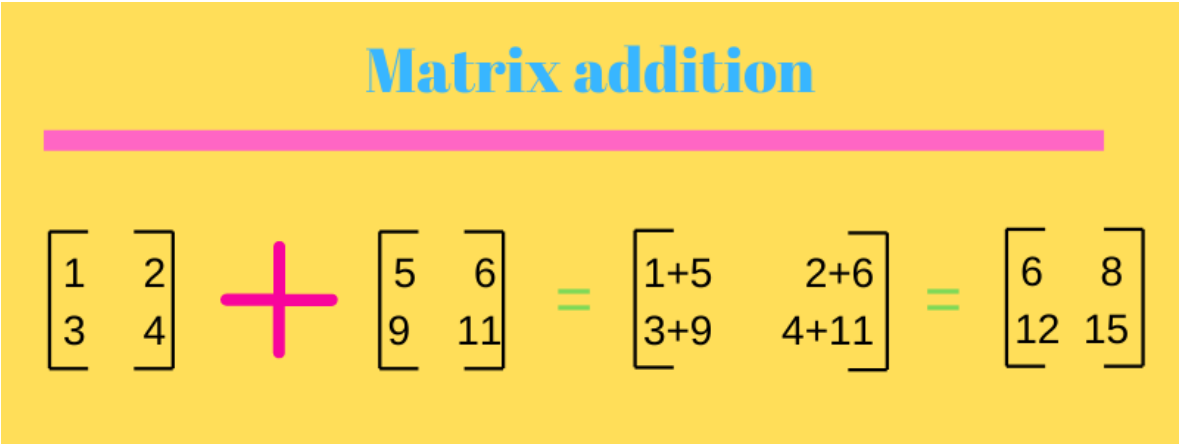
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
C:\desarrollo\Proyectos Python\Blog>python matrices.py
[5 7 9]
[1 3]
[7 5]
[1 3 5 7 9 7 5]
[5 7 9 7 5 3 1]
```

MATRICES

Grupos de variables dentro de filas y columnas, que suelen crecer de manera vertical, horizontal o de las 2 maneras.

En la siguiente imagen podemos ver 2 matrices que tiene 2 filas y 2 columnas, y se explica como se realiza la suma de matrices.



Teniendo en cuenta la teoría anterior, en la siguiente imagen, se muestra la creación de una matriz simple de 5 columnas por 3 filas, y este es un ejemplo de como se declara una matriz en el lenguaje de Python.

main.py

```python


matriz = [
    [85, 45, 1, 7, 9],
    [12, 52, 9, 151, 56],
    [76, 10, 56, 99, 9],
]

# Inicializar y asumir que tanto el mayor como
```

--VISUAL--

```
De la matriz:
[[85, 45, 1, 7, 9], [12, 52, 9, 151, 56], [76, 10, 56, 99, 9]]
El menor es 1 y el mayor es 151
>
```

Tipo de matices

Los tipos de matrices se dividen dependiendo sus características la división mas común es por su forma

Matriz por forma:

Estas pueden ser matriz por fila cuando es solo una fila o quizás matriz por columna cuando es una sola columna.

Matrices cuadradas: cuando la cantidad de filas es la misma que la cantidad de columnas.

Matriz en rectángulo: que es cuando hay mas filas que columnas o mas columnas que filas.

Matriz por valores:

Matriz nula (o cero): todos sus elementos son 0.

Matriz identidad (I): es cuadrada, con 1 en la diagonal principal y 0 en el resto.

Matriz diagonal: solo tiene valores distintos de 0 en la diagonal principal.

Matriz escalar: es diagonal y todos los valores de la diagonal son iguales.

Ejemplos de matrices

En la siguiente imagen, se muestra código en lenguaje C que crea 3 arreglos A, B y C y luego los recorre con un for para imprimir cada variable dentro del arreglo

```
1
2
3   #include <iostream>
4   #include "conio.h"
5
6   using namespace std;
7
8   int main(){
9
10      int A[3][3] = {{5,7,8},{5,7,8},{5,7,8}};
11      int B[3][3] = {{5,7,8},{5,7,8},{5,7,8}};
12      int C[3][3];
13
14      for(int i =0; i<3; i++){
15          for(int j =0; j<3; j++)
16              C[i][j] = A[i][j] - B[i][j];
17      }
18      for(int i =0; i<3; i++){
19          for(int j =0; j<3; j++)
20              cout << C[i][j] << "\t";
21
22          cout << endl;
23      }
24      _getch();
25  }
```

En la siguiente imagen se muestra lenguaje de programación C donde se crea un arreglo con poca información y con el ciclo for, se recorre cada variable dentro del arreglo.

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       int num[5]={0,0,0,0,0};
8       cout<<"Guarda 5 numeros"<<endl;
9       for (int i=0;i<=4;i++)
10      {
11          cin>>num[i];
12      }
13      cout<<"Los numeros son:"<<endl;
14      for (int i=0;i<=4;i++)
15      {
16          cout<<num[i]<<endl;
17      }
18      return 0;
19  }
```

En la siguiente imagen se muestra el ejemplo de crear un arreglo con las variables de cada letra del abecedario y como recorrerlo para mostrar cada variable por medio de un for

```
4   #include <iostream>
5   #include <stdlib.h>
6   #include "conio.h"
7
8   using namespace std;
9
10  int main() {
11
12      char matrizAlfabeto[10][10];
13      char alfabeto[] = {'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u',
14                         'v','w','x','y','z'};
15
16      for( int i = 0; i < 10; i++ ){
17          for( int j = 0; j < 10; j++ ){
18              matrizAlfabeto[i][j] = alfabeto[ rand() % 25 ];
19          }
20      }
21
22      for( int i = 0; i < 10; i++ ){
23          for( int j = 0; j < 10; j++ )
24              cout << matrizAlfabeto[i][j] << "\t";
25          cout << endl;
26      }
27      _getch();
28  }
```

El código que tenemos a continuación se muestra el java la creación de 2 matrices 3x3 y un ejemplo de sumatoria entre las 2 matrices, iterando cada fila y columna con su respectiva en la otra matriz.

```java
    public static void main(String[] args) {

        int[][] matriz1 = {
                {10, 5, 12},
                {98, 5, 6},
                {8, 4, 6},
        };
        int[][] matriz2 = {
                {8, 7, 5},
                {19, 56, 4},
                {8, 76, 90},
        };
        sumarMatrices(matriz1, matriz2);
    }
}
```

Run: Main

```
C:\jdk\jdk-16\bin\java.exe "-javaagent:C:\Program Files\JetBrains\Inte
            1                   2                Suma

-------------------------------------------------------------
   10     5    12  |     8     7     5  |    18    12    17  |
   98     5     6  |    19    56     4  |   117    61    10  |
    8     4     6  |     8    76    90  |    16    80    96  |

Process finished with exit code 0
```

En el siguiente código se muestra la creación e iniciación de una matriz con números random dentro de ella en la función inicializarmatriz y luego la función mostrarmatiz que recorre la matriz mostrando cada celda en pantalla. Esto realizado en java.

```java
import java.util.Scanner;

public class FuncionInicializarYMostrarMatrizJava {

        static Scanner entrada = new Scanner(System.in);

        static void inicializarMatriz(int [][] mat, int n)
        {
             for(int i= 0; i < n; i++){
                 for(int j= 0; j < n; j++)
                 mat[i][j] = (int)(Math.random() * 1001);
             }
        }
        static void mostrarMatriz(int [][] mat, int n)
        {
             for(int i= 0; i < n; i++){
                 for(int j= 0; j < n; j++)
                     System.out.print(mat[i][j] + "\t");
                 System.out.println();
             }
        }

        public static void main(String[] args) {
             // TODO code application logic here

             int n;

             System.out.print("Favor ingresar el valor de n: ");
             n=entrada.nextInt();

             int [][] mat = new int[n][n];

             inicializarMat  (mat, n)
             mostrarMatriz(  ,  );

        }
}
```

Otro ejemplo de creación de matriz en java 3x3 y 2 for que recorren la matriz por columna y fila en búsqueda de que muestre cada dato dentro de la matriz, de una manera sencilla

```java
Tutorial.java
public class Tutorial {

    public static void main(String [] Args){

        int matriz [][] = {{1,2,3},{4,5,6},{7,8,9}};

        for(int i =0;i < matriz.length;i++){
            for(int j =0; j < matriz[0].length;j++){
                System.out.print(matriz[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Problems | Javadoc | Declaration | Console

```
<terminated> Tutorial [Java Application] C:\Program Files\Java\jre7\bin\javaw.exe (Aug 1, 2012 4:17:02 PM)
1 2 3
4 5 6
7 8 9
```

En el siguiente código de Python se muestra como se combinan 2 matrices A y B con valores predefinidos y se crea la matriz C la cual se iprime en pantalla con el siguiente for de visualización.

```
>>> #HACEMOS LOS VALORES DE LA MATRIZ RESULTADO IGUAL A LA SUMA DE LAS 2 MATRICES.
>>> for i in range(filas):
        for j in range(colum):
                C[i][j]=A[i][j]+B[i][j]
```

```
>>> #VISUALIZAMOS MATRIZ RESULTANTE.
>>> for i in range(filas):
        R=[]
        for j in range(colum):
                R.append(C[i][j])
        print(R)
```

```
[2, 3, 4, 5]
[7, 4, 5, 2]
[9, 6, 4, 5]
[9, 10, 1, 4]
[5, 3, 2, 8]
>>>
```

El siguiente código de Python muestra como se le ingresan variables dentro de la matriz, recorriéndola por filas y columnas y un input en pantalla para ir dándole los datos.

```
#FUNCIÓN PARA DEFINIR MATRIZ
def crea_matriz(fil,col):
    f=-1;c=-1
    e_fil=[]
    for f in range(fil):
        e_col=[]
        f+=1
        for c in range(col):
            c+=1
            valor=OK(input('Introduzca el componente (%d,%d): '%(f,c)))
            e_col.append(valor)
        e_fil.append(e_col)
        matri=np.array(e_fil,float)
    return matri
```